Real-Time Schedulers for STM32: Performance Evaluation of EDF over Superloop

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. Introduction

STM32 is a family of microcontrollers developed by STMicroelectronics for embedded applications. These devices integrate a processor, memory, and peripheral interfaces on a single chip [1].

STM32-based devices support applications such as environmental monitoring [2] and equipment condition monitoring [3]. In these applications maintaining responsiveness as the workload increases is a key challenge.

The paper examines how the choice of scheduling approach affects responsiveness and processing cost on STM32. We compare two approaches: Superloop [4] and Earliest Deadline First (EDF) [5].

A. Related works

- Superloop in RTS.
- EDF and chunked EDF.
- Our evaluation.

B. Contribution

First, we compare Superloop and chunked EDF for periodic tasks with implicit deadlines on an STM32F767 microcontroller. Then, we evaluate the effects of task-set utilization on response time, deadline misses, skipped releases, and measured task-selection overhead. Also, we investigate how maximum chunk duration affects these metrics for chunked EDF. Both scheduler implementations are publicly available.

C. Paper organization

First, in Section II, we provide the necessary background on STM32. We describe the hardware platform in Section II-A and the project structure in Section II-B. Sections II-C and II-D explain how application functions execute and introduce the computation and sensor tasks used in our application. Next, in Section III we formalize the periodic task model used in our experiments. Finally, in Section IV, we describe Superloop, fully preemptive EDF, and chunked EDF and illustrate their schedules for the same task set.

II. Background on STM32

A. Hardware platform

We use an STM32F767 microcontroller with four sensors in our experiments (Fig. 1). The sensors collect different types of data and use different interfaces, as shown in Table I. The code that reads and processes their data forms the sensor-based workload for our scheduling experiments.

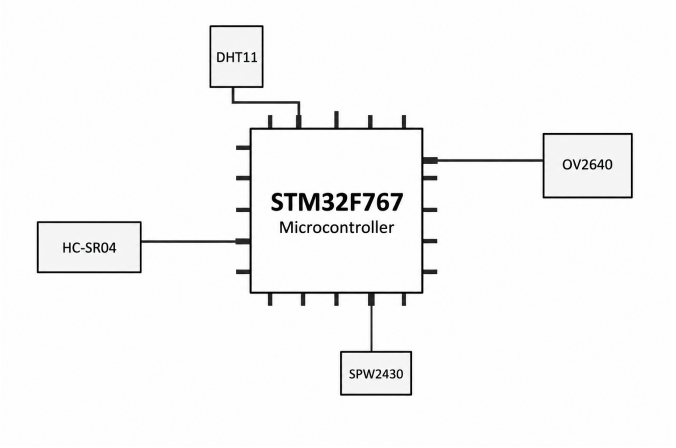


Fig. 1. Sensor platform based on STM32F767.

TABLE I
Sensor functions and interfaces.

Sensor	Purpose	Interface
DHT11 [6]	Temperature and humidity	Single-wire digital, GPIO
HC-SR04 [7]	Distance	Trigger/echo, GPIO
OV2640 [8]	Image acquisition	Parallel data, SCCB
SPW2430 [9]	Signal acquisition	Analog output, ADC

GPIO - General-Purpose Input/Output; SCCB - Serial Camera Control Bus; ADC - Analog-to-Digital Converter.

B. STM32 Project Structure

We develop the code for this platform in a C project generated by STM32CubeMX. STM32CubeMX configures the microcontroller pins and peripherals and generates their initialization code. The project is edited, built, and debugged in STM32CubeIDE.

Fig. 2 shows the main project components. Application source files and headers are placed in `Core/Src` and `Core/Inc`, respectively. The source files include `main.c`, which contains system initialization and the main loop from which application functions are called.

C. Default execution model

The generated `main()` function first initializes the system and peripherals, then enters an infinite `while (1)` loop (Listing 1). STM32CubeMX marks `USER CODE` regions where developers can add their own code without losing it

```
test_project/
├── Core/ ..... Application code
│   ├── Inc/ ..... Header files
│   └── Src/ ..... Source files
│       ├── main.c ..... Init. and main loop
│       ├── stm32f7xx_it.c ..... Interrupt handlers
│       ├── stm32f7xx_hal_msp.c ..... Low-level peripheral setup
│       └── system_stm32f7xx.c ..... Core init. and clock state
│   └── Startup/ ..... Startup code and interrupt vectors
│       └── startup_stm32f767zitx.s
├── Drivers/ ..... Hardware support
│   ├── CMSIS/ ..... Core and MCU definitions
│   └── STM32F7xx_HAL_Driver/
├── STM32F767ZITX_FLASH.ld
├── STM32F767ZITX_RAM.ld
└── test_project.ioc ..... CubeMX configuration
```

Fig. 2. Main components of the STM32CubeMX-generated project.

Listing 1. Simplified STM32CubeMX-generated main function showing system initialization, user-code regions, and the infinite loop.

```
int main(void) {
  /* CubeMX-generated initialization */
  MPU_Config();
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  /* Other peripheral initialization */

  /* USER CODE BEGIN 2 */
  /* User-defined application setup */
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    /* User-defined functions */
  }
  /* USER CODE END 3 */
}
```

when the project is generated again. By default, the loop contains no application functions.

Application functions can be called from inside this loop. In a Superloop, these functions run in a fixed order, and the sequence repeats after the last function returns [4] (Fig. 3). For example, `taskB()` runs only after `taskA()` returns, so the execution time of one function affects when the next function can start. In our application, such functions perform computations or read sensor data.

D. User defined tasks

1) Synthetic tasks: We use synthetic tasks to create CPU workload without reading data from sensors. Listing 2 shows one example, where a function updates a stored value through 10,000 iterations. The number of iterations can be changed to control the amount of computation, although the actual execution time must be measured on the STM32.

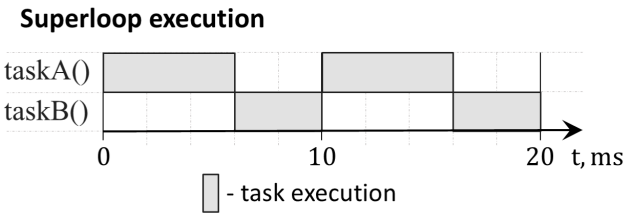


Fig. 3. Example of Superloop execution. Functions are executed sequentially in a fixed order, and the loop repeats indefinitely.

Listing 2. Example of a synthetic computation.

```
static volatile uint32_t state = 1U;

void SyntheticTask_Run(void)
{
  uint32_t value = state;

  for (uint32_t i = 0; i < 10000U; i++) {
    value = value * 1664525U + i;
  }

  state = value;
}
```

2) Sensor tasks: Unlike synthetic tasks, sensor tasks must communicate with external devices. Listing 3 shows how the STM32 reads data from a DHT11 sensor. The function first holds the data pin low for 30 ms and then reads the sensor response. With this blocking implementation, the main loop waits for the sensor read to complete before calling another function

Sensor readings also have timing limits. The DHT11 requires at least 1 second between readings [6], while the HC-SR04 recommends more than 60 ms between measurements [7]. Calling these functions on every pass through the main loop may violate these limits. Instead, we call sensor-reading functions at set intervals [10]. The next section introduces the periodic task model used in our experiments.

Listing 3. Example of a DHT11 sensor read operation.

```
int DHT11_Read(uint8_t *temp, uint8_t *hum)
{
  DHT11_SetOutput();

  HAL_GPIO_WritePin(DHT11_PORT,
  DHT11_PIN, GPIO_PIN_RESET);

  HAL_Delay(30);

  return DHT11_ReadTransaction(temp, hum);
}
```

III. System Model

We use the periodic task model of Liu and Layland [5] and adapt the task and release notation of Burmyakov and Bini [11]. Table II summarizes the notation.

A real-time system $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ consists of n periodic tasks executed on one core ($m = 1$). Each task $\tau_i = (C_i, D_i, P_i)$ is defined by the worst-case execution time

C_i of a job, a relative deadline D_i, and a period P_i. We consider implicit deadlines, $D_i = P_i$.

Let $J_{i,j}$ denote the j-th job of τ_i, released at $r_{i,j}$. Consecutive jobs are released one period apart:

$$r_{i,j+1} = r_{i,j} + P_i. \tag{1}$$

Each job $J_{i,j}$ has an absolute deadline $d_{i,j} = r_{i,j} + D_i$. If completed, its finish time is denoted by $f_{i,j}$.

TABLE II
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Worst-case execution time of a job of τ_i
P_i	Task period
$D_i = P_i$	Relative deadline, equal to the period
$J_{i,j}$	The j-th job of task τ_i
$r_{i,j}$	Release time of job $J_{i,j}$
$f_{i,j}$	Job completion time
$d_{i,j} = r_{i,j} + D_i$	Absolute deadline of job $J_{i,j}$
m	Number of cores
$U_i = C_i/P_i$	Task utilization
$U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$	Task-set utilization

- A schedule specifies which job executes at each time, if any.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ misses its deadline if $f_{i,j} > d_{i,j}$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$.

IV. SCHEDULERS

We compare two scheduling approaches: a periodic Superloop and chunked Earliest Deadline First (EDF). We also show fully preemptive EDF for comparison.

A. Superloop

We use a periodic Superloop as a non-preemptive scheduler that checks tasks in a fixed order [10]. Each task runs only when its next release time has been reached [12]. Once started, a task runs to completion before the next task is checked. Algorithm 1 shows this process.

Algorithm 1 Superloop Scheduler

```
function SUPERLOOP(T)
    t_i^next ← 0,  i = 1, ..., n                ▷ Initial releases
    while true do
        for i ← 1 to n do                        ▷ Fixed task order
            t^cur ← NOW()                        ▷ Read current system time
            if t_i^next ≤ t^cur then
                J_{i,j} ← RELEASEJOB(τ_i, t_i^next)
                EXECUTE(J_{i,j})                 ▷ Non-preemptive
                f_{i,j} ← NOW()
                t_i^next ← t_i^next + P_i
                while t_i^next < f_{i,j} do t_i^next ← t_i^next + P_i
            end if
        end for
    end while
end function
```

Fig. 4(a) illustrates the Superloop schedule for tasks τ_1 and τ_2. Both tasks release their first jobs at $t = 0$. Since τ_1 comes first in the polling order, $J_{1,1}$ executes until $t = 18$ ms. Job $J_{2,1}$ then starts and completes at $f_{2,1} = 28$ ms, missing its deadline $d_{2,1} = 20$ ms.

The release at $r_{2,2} = 20$ ms occurs while $J_{2,1}$ is still running. Once this job completes, the scheduler counts that release as skipped and advances the next release time to $r_{2,3} = 40$ ms. Polling continues with no ready job until then.

B. Earliest Deadline First

In contrast, fully preemptive EDF selects the ready job with the earliest absolute deadline. A newly released job preempts the running job if its deadline is earlier.

Fig. 4(b) shows the EDF schedule for the same task set. At $t = 0$, $J_{2,1}$ has the earlier deadline and executes until $t = 10$ ms. Job $J_{1,1}$ then starts, but is preempted at $r_{2,2} = 20$ ms because the new job has a deadline $d_{2,2} = 40$ ms, earlier than $d_{1,1} = 50$ ms. Job $J_{2,2}$ executes until $t = 30$ ms, after which $J_{1,1}$ resumes and completes at $t = 38$ ms. The next job of τ_2 executes from $t = 40$ to $t = 50$ ms. Thus, no deadline is missed and no release is skipped in this example.

C. Chunked Earliest Deadline First

Unlike classical EDF, our chunked EDF implementation executes jobs in non-preemptive chunks of at most Δ ms. Sensor operations are split only at points that preserve the timing requirements of the interface.

Algorithm 2 outlines chunked EDF. The scheduler maintains a set $\mathcal{R}$ of released but unfinished jobs. After each chunk, it updates $\mathcal{R}$ with newly released jobs and selects the job with the earliest absolute deadline.

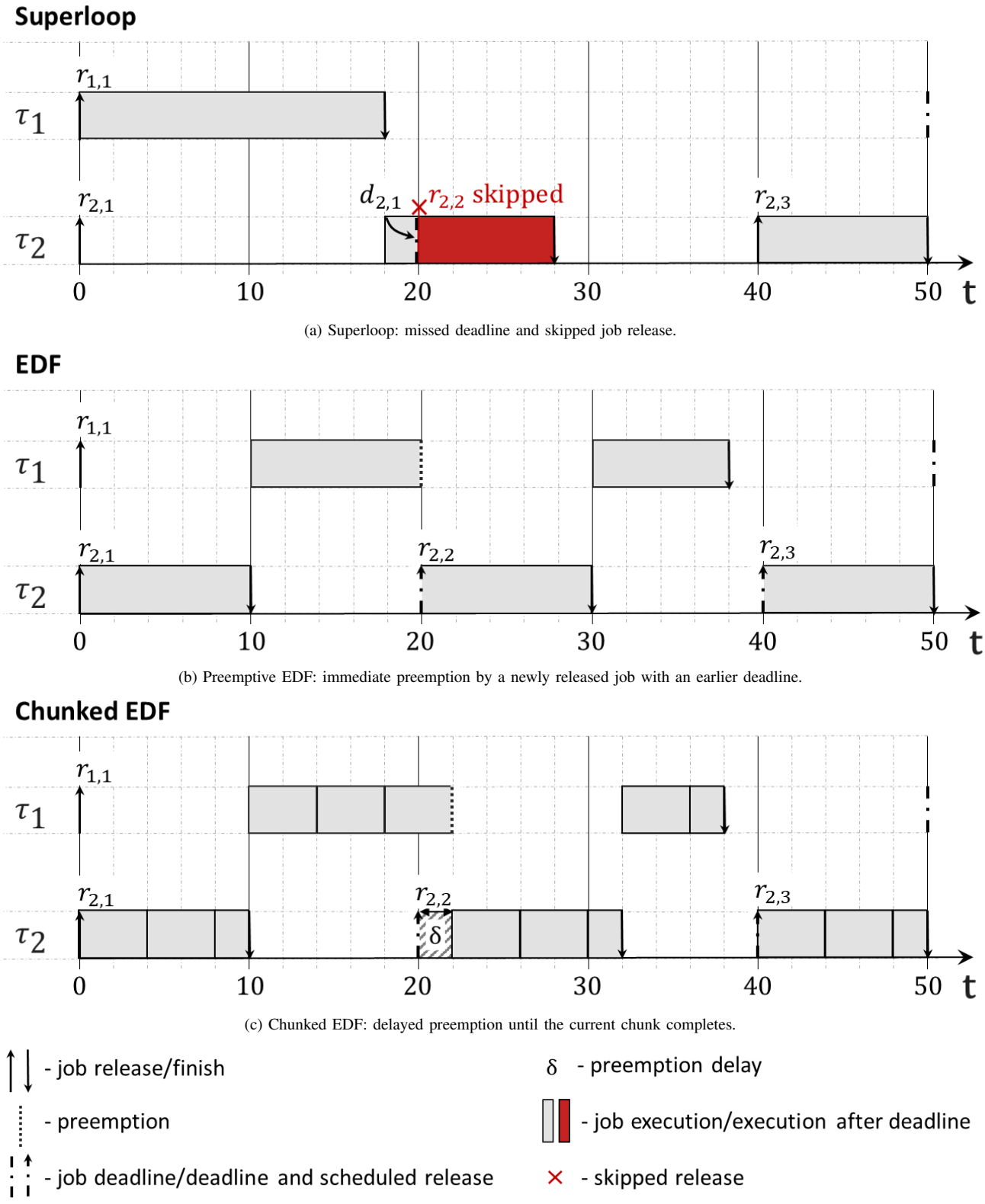
Fig. 4(c) shows the same task set with $\Delta = 4$ ms. At $r_{2,2} = 20$ ms, the first job of τ_1 is still executing a chunk, which ends at $t = 22$ ms. The new job of τ_2 therefore starts at $t = 22$ ms, with a preemption delay of $\delta = 2$ ms. It executes in chunks of 4, 4, and 2 ms and completes at $t = 32$ ms. The shorter final chunk reflects the remaining work. The interrupted job of τ_1 then resumes and completes at $t = 38$ ms.

Algorithm 2 Chunked EDF Scheduler

function CHUNKEDEDF($\mathcal{T}, \Delta$)
  $\mathcal{R} \leftarrow \emptyset$ ▷ Set of unfinished jobs
  $t_i^{\mathsf{next}} \leftarrow 0, \quad i = 1, \ldots, n$ ▷ Initial releases
  while true **do**
    $t^{\mathsf{cur}} \leftarrow$ NOW() ▷ Read current system time
    for all $\tau_i \in \mathcal{T}$ **do** ▷ Check releases
      if $t_i^{\mathsf{next}} \leq t^{\mathsf{cur}}$ and no τ_i job in $\mathcal{R}$ **then**
        $J_{i,j} \leftarrow$ RELEASEJOB($\tau_i, t_i^{\mathsf{next}}$)
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{J_{i,j}\}$ ▷ Add new job
      end if
    end for
    if $\mathcal{R} \neq \emptyset$ **then** ▷ Pending work
      $J_{i,j} \leftarrow$ SELECTEDF($\mathcal{R}$) ▷ Earliest deadline
      EXECUTECHUNK($J_{i,j}, \Delta$) ▷ Duration $\leq \Delta$
      if COMPLETED($J_{i,j}$) **then**
        $f_{i,j} \leftarrow$ NOW()
        $t_i^{\mathsf{next}} \leftarrow t_i^{\mathsf{next}} + P_i$
        while $t_i^{\mathrm{next}} < f_{i,j}$ **do** $t_i^{\mathrm{next}} \leftarrow t_i^{\mathrm{next}} + P_i$
        $\mathcal{R} \leftarrow \mathcal{R} \setminus \{J_{i,j}\}$ ▷ Remove completed job
      end if
    end if
  end while
end function

$$\tau_i = (C_i, D_i, P_i), \qquad D_i = P_i, \qquad m = 1$$
$$\tau_1 = (18, 50, 50), \qquad \tau_2 = (10, 20, 20)$$

All times in ms. Maximum chunk duration in (c): $\Delta = 4$ ms.



(a) Superloop: missed deadline and skipped job release.

(b) Preemptive EDF: immediate preemption by a newly released job with an earlier deadline.

(c) Chunked EDF: delayed preemption until the current chunk completes.

Fig. 4. Comparison of Superloop, preemptive EDF, and chunked EDF schedules for the same periodic task set.

V. Performance evaluation

A. Synthetic workload

B. Real sensors workload

References

[1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf

[2] STMicroelectronics, "Process and environment monitoring," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/process-and-environment-monitoring.html

[3] ——, "Condition monitoring / predictive maintenance," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/condition-monitoring-predictive-maintenance.html

[4] Arm Keil, "Execution models: Single task system (Superloop)," accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[6] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[7] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[8] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf

[9] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/218/SPW2430HR5H-B-1290924.pdf

[10] P. Koopman, "Multi-rate main loop tasking," Dec. 2015, accessed: Oct. 19, 2026. [Online]. Available: https://betterembsw.blogspot.com/2015/12/multi-rate-main-loop-tasking.html

[11] A. Burmyakov and E. Bini, "Improving the efficiency of global EDF exact schedulability test," *IEEE Transactions on Computers*, pp. 1–14, 2026.

[12] N. Jones. (2024) You don't need an RTOS (part 1). Accessed: Oct. 19, 2026. [Online]. Available: https://www.embeddedrelated.com/showarticle/1636.php